OPERATING SYSTEMS MINI PROJECT

CPU SCHEDULING SIMULATOR

Submitted by: Samruddhi Narayan Thite

Roll No.: 43

Department: Artificial Intelligence and Data Science

College: Alard University Pune

Academic Year: 2026-27

Abstract

CPU scheduling is an important function of an operating system. It selects a ready process to use the CPU. This project implements FCFS, non-preemptive Shortest Job First (SJF), non-preemptive Priority Scheduling and Round Robin in C. The program accepts process arrival time, burst time and priority, then displays a Gantt chart, completion time, turnaround time, waiting time and average values.

1. Introduction

In a multiprogramming operating system, multiple processes may be ready to execute. The CPU scheduler chooses the next process. Different algorithms aim to improve waiting time, fairness or responsiveness. This simulator helps students compare common algorithms using the same input data.

2. Problem Statement

To develop a menu-driven C program that simulates CPU scheduling algorithms and calculates their execution order and performance metrics.

3. Aim

To study and simulate common CPU scheduling algorithms using C programming.

4. Objectives

• Implement FCFS, SJF, Priority and Round Robin. • Accept arrival time, burst time and priority. • Display a Gantt chart. • Calculate CT, TAT, WT and averages. • Compare the behavior of different scheduling policies.

5. Scope

This is an educational simulation and does not change the scheduling policy of the computer's actual operating system. SJF and Priority are non-preemptive in this version. Round Robin uses a user-defined positive time quantum.

6. Theory and Formulae

Arrival Time (AT): when a process enters the ready queue. **Burst Time (BT):** CPU time required. **Completion Time (CT):** time when execution finishes. **Turnaround Time (TAT) = CT − AT. Waiting Time (WT) = TAT − BT.**

FCFS: Executes processes in arrival order. **SJF:** Chooses the arrived process with the smallest burst time. **Priority:** Chooses the arrived process with the highest priority; in this project, a smaller priority number means higher priority. **Round Robin:** Each ready process receives CPU time up to the time quantum; unfinished processes return to the queue.

7. Requirements

Hardware: Standard desktop or laptop. **Software:** Windows or Linux, GCC or another C compiler, and an editor such as VS Code or Code::Blocks.

8. System Design

Inputs include process count, arrival time, burst time, priority and algorithm selection. Round Robin additionally requires a time quantum. The program schedules processes, records completion times, calculates metrics and displays a Gantt chart and average values.

9. Algorithm

1. Start.

2. Read and validate the number of processes.

3. Read arrival time, positive burst time and non-negative priority for every process.

4. Display the menu and read the selected algorithm.

5. Select processes according to the scheduling policy and record their execution intervals.

6. For Round Robin, execute each process for the smaller of its remaining burst time and the time quantum; requeue unfinished processes.

7. Record completion times.

8. Calculate TAT = CT − AT and WT = TAT − BT.

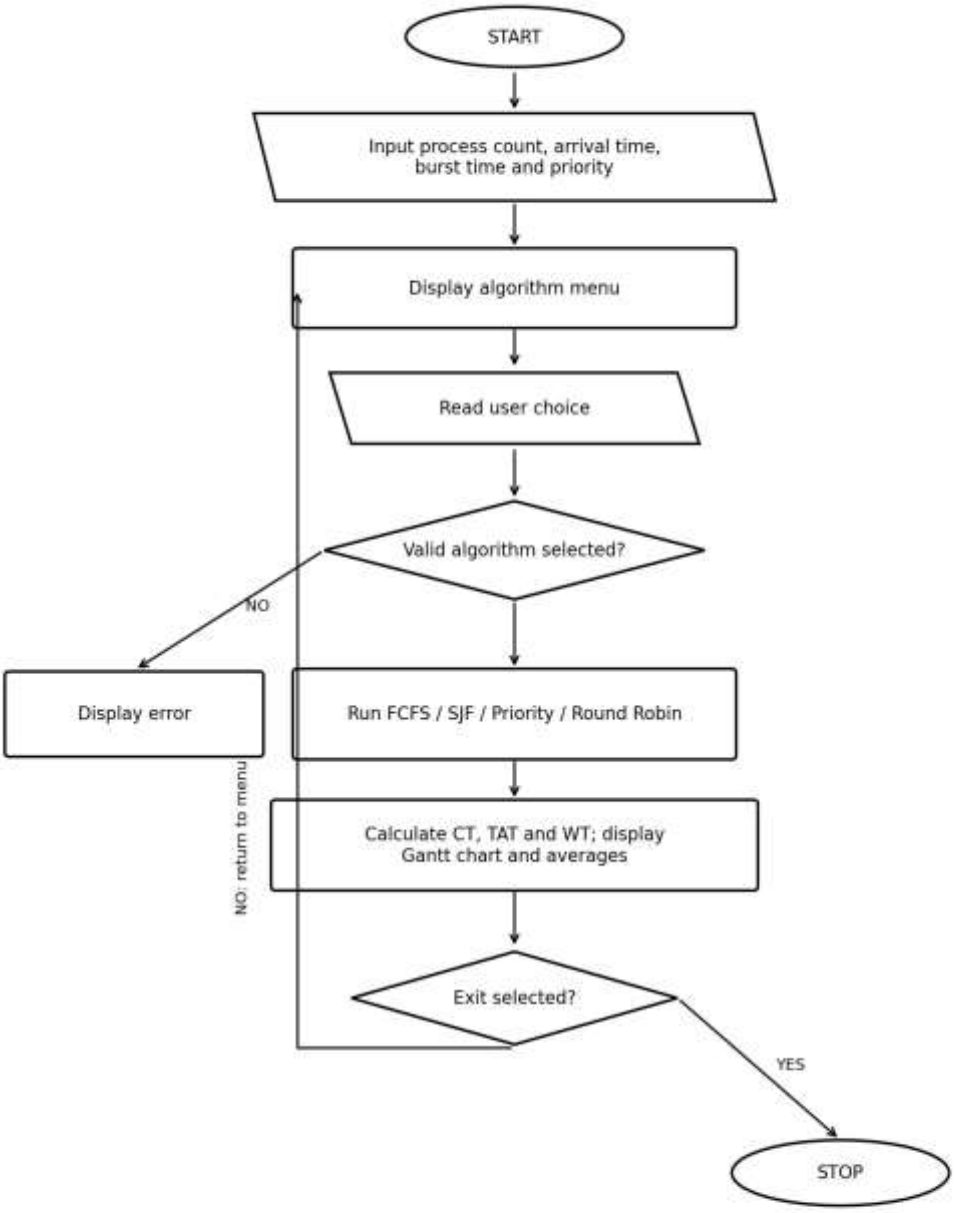
9. Display the Gantt chart and metrics.

10. Repeat until Exit is selected.

11. Stop.

10. Flowchart

Main menu flow of the simulator:



11. Source Code and Execution

The complete program is supplied in the separate file **cpu_scheduling.c**. Compile using: `gcc cpu_scheduling.c -o cpu_scheduling`. On Linux run `./cpu_scheduling`; on Windows run `cpu_scheduling.exe`.

12. Sample Input

Process	Arrival	Burst	Priority
P0	0	5	2
P1	1	3	1
P2	2	8	3
P3	3	6	2

Choose option 5 to run all algorithms, then enter time quantum 2 for Round Robin.

13. Testing

Test 1: choose FCFS; a Gantt chart and metrics should appear. Test 2: choose SJF; the shortest arrived job should run next. Test 3: choose Priority; the smallest priority number should run first among arrived processes. Test 4: choose Round Robin with quantum 2; unfinished jobs should be requeued. Test 5: invalid process count, negative arrival time, non-positive burst time or non-positive quantum should be rejected.

14. Results and Discussion

The program demonstrates how execution order changes with the selected policy. It calculates CT, TAT and WT for each process and the average waiting and turnaround times. The best algorithm depends on the workload; no single algorithm is optimal for every input.

15. Advantages

• Menu-driven and easy to use. • Compares four standard algorithms. • Gantt chart makes execution order visible. • Calculates per-process and average performance metrics.

16. Limitations

• SJF and Priority are non-preemptive. • Priority aging is not implemented, so starvation may occur. • A fixed set of processes is entered at program start. • The Gantt chart is text-based.

17. Future Scope

The project can be extended with preemptive SJF, preemptive Priority, a graphical interface, output-file generation and automatic comparison charts.

18. Conclusion

The CPU Scheduling Simulator provides a practical demonstration of FCFS, SJF, Priority and Round Robin. It helps students understand how scheduling decisions affect completion, waiting and turnaround times and reinforces core C programming and Operating Systems concepts.

19. References

• Silberschatz, Galvin and Gagne, *Operating System Concepts*, Wiley. • William Stallings, *Operating Systems: Internals and Design Principles*, Pearson. • Operating Systems lecture notes and laboratory manual provided by the department.

Submission note: Fill in the cover page details and follow your department's report format before submitting.